

安徽大学《深度学习与神经网络》

实验报告 2

学号： WA2224013 专业： 机器人工程 姓名： 郭义月
实验日期： 2025.5.25 教师签字： _____ 成绩： _____

[实验名称] PyTorch 框架深度学习基础实验

[实验目的]

1. 熟悉和掌握 PyTorch 层和块的定义和设计
2. 熟悉和掌握 PyTorch 数据、模型的保存和重载
3. 熟悉和掌握 PyTorch 模型构建和训练的完整流程

[实验要求]

1. 采用 Python 语言基于 PyTorch 深度学习框架进行编程
2. 代码可读性强：变量、函数、类等命名可读性强，包含必要的注释
3. 提交实验报告要求：
 - 命名方式：“学号-姓名-Lab-N” (N 为实验课序号，即：1-6)；
 - 截止时间：下次实验课当晚 23:59；
 - 提交方式：智慧安大-网络教育平台-作业；
 - 按时提交（**过时不补**）；

[实验内容]

1. 层与块

- 学习、运行和调试参考教材 5.1 小节内容
- 完成练习题 1,2

2. 参数管理：

- 学习、运行和调试参考教材 5.2 小节内容
- 完成练习题 1（题目中 FancyMLP 改为 MLP）和练习 3

3. 自定义层：

- 学习、运行和调试参考教材 5.4 小节内容
- 完成练习题 1

4. 读写文件：

- 学习、运行和调试参考教材 5.5 小节内容
- 完成练习题 2（请写代码举例完成）

5. 构建含有三个隐藏层的感知机网络对 Fashion-MNIST 的分类

- 隐藏层神经元个数分别设为 256,128；采用交叉熵损失
- 画出训练损失、训练正确率、测试正确率随训练轮次 Epoch 的变化曲线
- 尝试不同的激活函数，观察和分析实验结果
- 尝试再网络中加入 dropout 层，观察和分析实验结果

6. 参考资料：

- 参考教材：<https://zh-v2.d2l.ai/d2l-zh-pytorch.pdf>
- PyTorch 官方文档：<https://pytorch.org/docs/2.0/>；
- PyTorch 官方论坛：<https://discuss.pytorch.org/>

[实验代码和结果]

实验 1：层与块，学习 5.1

在我们写代码时，经常研究讨论“比单个层大”但“比整个模型小”的组件，也就是块，块（block）可以描述单个层、由多个层组成的组件或整个模型本身。使用块进行抽象的一个好处是可以将一些块组合成更大的组件。

我们可以先用自带的方法实现包含一个具有 256 个单元和 ReLU 激活函数的全连接隐藏层，然后是一个具有 10 个隐藏单元且不带激活函数的全连接输出层。

```
> net = nn.Sequential(nn.Linear(20,256),nn.ReLU(), nn.Linear(256,10))
X = torch.rand(3,20)
net(X) # net.__call__(X)的缩写，可以直接执行传播函数
[7] ✓ 0.0s
tensor([[ -0.0862,  0.0693, -0.1162, -0.0238,  0.0015, -0.0571,  0.0261, -0.0841,
          0.0670,  0.0230],
        [-0.0987,  0.0098, -0.1751, -0.1613, -0.0363, -0.0111, -0.0695,  0.0181,
          -0.0122, -0.0644],
        [-0.1958, -0.1035, -0.0744,  0.0531, -0.0951, -0.1124, -0.0619, -0.1909,
          0.1121, -0.0881]], grad_fn=<AddmmBackward0>)
```

如上图所示，最后输出 3×10 的张量。通过实例化 `nn.Sequential` 来构建我们的模型，层的执行顺序是作为参数传递的。简而言之，`nn.Sequential` 定义了一种特殊的 Module，即在 PyTorch 中表示一个块的类，它维护了一个由 Module 组成的有序列表。注意，两个全连接层都是 `Linear` 类的实例，`Linear` 类本身就是 Module 的子类。

5.1.1 自定义块

从零开始编写的一个块包含一个多层感知机，其具有 256 个隐藏单元的隐藏层和一个 10 维输出层。我们定义的 MLP 类继承了表示块的类。我们的实现只需要提供我们自己的构造函数（Python 中的 `__init__` 函数）和前向传播函数。

层的执行顺序是作为参数传递的，`nn.Sequential` 维护了由 Module 组成的有序列表

两个全连接层都是 `Linear` 的实例

这个前向传播函数将列表中的每个块连接在一起，将每个块的输出作为下一个块的输入

```
class MLP(nn.Module):
    # 模型参数声明层，这里我们声明两个全连接层
    def __init__(self):
        # 调用父类Module的初始化函数来必要的初始化，这样在类实例化时也可以指定别的参数
        super().__init__()
        self.hidden = nn.Linear(20, 256)
        self.output = nn.Linear(256, 10)
    def forward(self, X):
        return self.output(F.relu(self.hidden(X))) # 先hidden, 再relu, 再out
[7] ✓ 0.0s
```

结果输出 $3 * 10$ 的张量：

```
net = MLP()
net(X)

✓ 0.0s

tensor([[ 0.0654, -0.0458, -0.0947,  0.0653, -0.2166, -0.0903, -0.1728,  0.0696,
          -0.0117, -0.1129],
        [ 0.0563, -0.1492, -0.1418,  0.1001, -0.0835, -0.0116, -0.2047,  0.1149,
          0.0584, -0.1427],
        [ 0.1234, -0.0871, -0.1474, -0.0831, -0.1948,  0.0337, -0.1580,  0.0324,
          0.0157,  0.0140]], grad_fn=<AddmmBackward0>)
```

块的一个主要优点是它的多功能性。我们可以子类化块以创建层（如全连接层的类）、整个模型（如 MLP 类）或具有中等复杂度的各种组件。

5.1.2 顺序块

想要自定义 Sequential 类需要先明白 Sequential 类是如何工作的。

Sequential 的设计是为了把其他模块串起来。所以构建我们自己的简化的 MySequential 只需要定义两个关键函数：

1. 一种将块逐个追加到列表中的函数；
2. 一种前向传播函数，用于将输入按追加块的顺序传递给块组成的“链条”

顺序块 需要定义两个关键函数：

一种将块逐个加到列表中的函数；

一种前向传播函数，用于将输入按追加块的顺序传递给块组成的链条

```
class MySequential(nn.Module):
    def __init__(self, *args):
        super().__init__()
        for idx, module in enumerate(args):
            self._modules[str(idx)] = module # _module 的类型是 OrderedDict
    def forward(self, X):
        for block in self._modules.values():
            X = block(X)
        return X

✓ 0.0s
```

当 MySequential 的前向传播函数被调用时，每个添加的块都按照它们被添加的顺序执行。实验结果输出为 $3 * 10$ 的张量。

```

net = MySequential(nn.Linear(20, 256), nn.ReLU(), nn.Linear(256, 20))
net(X)
177 ✓ 0.0s
** tensor([[ -0.1352,  0.1426, -0.0326, -0.1378,  0.1183, -0.1640,  0.1481, -0.1845,
           0.0263,  0.3483,  0.0086, -0.1525,  0.1198, -0.1203, -0.1299,  0.0503,
          -0.2082, -0.2358,  0.3186,  0.0794],
          [ 0.0730,  0.0893,  0.0381,  0.0360, -0.0731, -0.1824,  0.1922, -0.1139,
          -0.0317,  0.3662, -0.0792, -0.1126, -0.0296, -0.2089, -0.1607,  0.0341,
          -0.2134, -0.1536,  0.3957,  0.0318],
          [-0.0251,  0.0470, -0.0282, -0.1048,  0.0083, -0.1885,  0.0211, -0.0579,
          -0.0280,  0.2392, -0.0294, -0.1091,  0.0060, -0.2338, -0.0781,  0.0176,
          -0.1909, -0.1013,  0.2353,  0.0778]], grad_fn=<AddmmBackward0>)

```

5.1.3 在前向传播函数中执行代码

当希望在前向传播函数中执行任意的数学运算，而不是简单地依赖预定义的神经网络层时，或者希望合并既不是上一层的结果也不是可更新参数的项，此时就需要在前向传播函数中执行代码。

在前向传播函数中执行代码

```

class FixedHiddenMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.rand_weight = torch.rand((20, 20), requires_grad=False)
        self.linear = nn.Linear(20, 20)
    def forward(self, X):
        X = self.linear(X)
        X = F.relu(torch.mm(X, self.rand_weight) + 1)
        X = self.linear(X) # 复用全连接层。这相当于两个全连接层共享参数
        while X.abs().sum() > 1: # 执行我们需要的逻辑
            X /= 2
        return X.abs().sum()
41 ✓ 0.0s

net = FixedHiddenMLP()
net(X)
53 ✓ 0.0s
tensor(0.5998, grad_fn=<SumBackward0>)

```

在 FixedHiddenMLP 模型中，有一个隐藏层，其权重（self.rand_weight）在实例化时被随机初始化，之后为常量。这个权重不是一个模型参数，因此它永远不会被反向传播更新。然后，神经网络将这个固定层的输出通过一个全连接层。注意，在返回输出之前，还运行了一个 while 循环，在 $L1$ 范数大于 1 的条件下，将输出向量除以 2，直到它满足条件为止。最后，模型返回了 X 中所有项的和。类似的方法，我们就可以直线将任意代码集成到神经网络计算的流程中。

混合搭配:

混合搭配组合块

```
class NestMlp(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(20, 64), nn.ReLU(), nn.Linear(64, 32), nn.ReLU())
        self.linear = nn.Linear(32, 16)
    def forward(self, X):
        return self.linear(self.net(X))

chimera = nn.Sequential(NestMlp(), nn.Linear(16, 20), FixedHiddenMlp())
chimera(X)
```

✓ 0.0s

tensor(0.8644, grad_fn=<SumBackward0>)

分析可知，此处的逻辑为，X 为 3×20 ，先经过 20×64 FC，再 ReLU，再 64×32 FC，再 ReLU，再 32×16 FC，再 16×20 FC，再 20×20 FC，再权重 20×20 相乘，每个值再+1 后 ReLU，再 20×20 FC，之后运算 $L1$ 范数大于 1 的条件下，将输出向量除以 2，直到它满足条件为止，最后返回了 X 中所有项的和。

实验 2：参数管理，学习 5.2

5.2.1 参数访问

先定义一个有三层的 net，可以通过 state_dict 访问

```
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 2))
X = torch.rand(2,4)
net(X)
```

✓ 0.0s

tensor([[0.1848, 0.1289],
 [0.0404, 0.2804]], grad_fn=<AddmmBackward0>)

参数访问

```
print(net[2].state_dict())
```

✓ 0.0s

OrderedDict([('weight', tensor([[-0.3411, 0.3368, -0.1338, -0.3418, 0.2566, 0.0558, 0.3452, -0.0008],
 [0.1943, -0.2471, 0.2449, 0.1372, -0.1093, 0.2314, -0.2334, -0.3120]])), ('bias', tensor([0.2852, 0.1506]))])

还可以访问目标参数，输出类型和值

目标参数

```
print(type(net[2].bias))      偏置的类型
print(net[2].bias)
print(net[2].bias.data)
print(net[2].weight.data)
```

✓ 0.0s

<class 'torch.nn.parameter.Parameter'>

Parameter containing:

tensor([0.2852, 0.1506], requires_grad=True)

tensor([0.2852, 0.1506])

tensor([[-0.3411, 0.3368, -0.1338, -0.3418, 0.2566, 0.0558, 0.3452, -0.0008],
 [0.1943, -0.2471, 0.2449, 0.1372, -0.1093, 0.2314, -0.2334, -0.3120]])

在网络中，由于我们还没有调用反向传播，所以参数的梯度处于初始状态。

```
net[2].weight.grad == None
```

```
True
```

一次性访问所有参数：

当我们需要对所有参数执行操作时，逐个访问它们可能会很麻烦。当我们处理更复杂的块（例如，嵌套块）时，情况可能会变得特别复杂，因为我们需要递归整个树来提取每个子块的参数。

```
一次性访问所有参数
```

```
# 访问第一个连接层
print(*[(name, param.shape) for name, param in net[0].named_parameters()])
# 访问所有层
print(*[(name, param.shape) for name, param in net.named_parameters()])
```

```
('weight', torch.Size([8, 4])) ('bias', torch.Size([8]))
('0.weight', torch.Size([8, 4])) ('0.bias', torch.Size([8])) ('2.weight', torch.Size([2, 8])) ('2.bias', torch.Size([2]))
```

```
net.state_dict()['2.bias'].data
```

```
tensor([0.2852, 0.1506])
```

从嵌套块收集参数：

如果多个块相互嵌套，可以通过如下方法收集参数。我们首先定义一个生成块的函数（可以说是“块工厂”），然后将这些块组合到更大的块中。

```
从嵌套块中收集参数
```

```
def block1():
    return nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 4), nn.ReLU())

def block2():
    net = nn.Sequential()
    for i in range(4):
        net.add_module(f'block{i}', block1())
    return net
rgnet = nn.Sequential(block2(), nn.Linear(4,1))
rgnet(X)
```

```
tensor([[-0.4938],
        [-0.4939]], grad_fn=<AddmmBackward0>)
```

可以 print 查看工作方式

```
print(rngnet)

Sequential(
  (0): Sequential(
    (block0): Sequential(
      (0): Linear(in_features=4, out_features=8, bias=True)
      (1): ReLU()
      (2): Linear(in_features=8, out_features=4, bias=True)
      (3): ReLU()
    )
    (block1): Sequential(
      (0): Linear(in_features=4, out_features=8, bias=True)
      (1): ReLU()
      (2): Linear(in_features=8, out_features=4, bias=True)
      (3): ReLU()
    )
    (block2): Sequential(
      (0): Linear(in_features=4, out_features=8, bias=True)
      (1): ReLU()
      (2): Linear(in_features=8, out_features=4, bias=True)
      (3): ReLU()
    )
    (block3): Sequential(
      (0): Linear(in_features=4, out_features=8, bias=True)
      (1): ReLU()
      (2): Linear(in_features=8, out_features=4, bias=True)
      (3): ReLU()
    )
  )
)
```

因为层是分层嵌套的，所以也可以像通过嵌套列表索引一样访问它们。

```
print(f'第一个块中的第二个子块中的第一个偏置: {rngnet[0][1][0].bias.data}')
print(f'第一个块中的第二个子块中的第一个偏置: {rngnet[0][1][0].weight.data}')
print(f'第二个块中的权重: {rngnet[1].weight.data}')
print(f'第二个块中的偏置: {rngnet[1].bias.data}')

第一个块中的第二个子块中的第一个偏置: tensor([-0.0997,  0.0693,  0.4464,  0.3611, -0.0806, -0.2656, -0.3685, -0.4596])
第一个块中的第二个子块中的第一个偏置: tensor([[ 0.3743,  0.2321, -0.3812, -0.0836],
        [ 0.2376,  0.1854, -0.3021, -0.4010],
        [ 0.2320,  0.2329, -0.1690, -0.0795],
        [-0.3488, -0.0216,  0.0243,  0.2503],
        [-0.1941,  0.0549, -0.4368,  0.2317],
        [-0.1023,  0.4305, -0.3575,  0.3646],
        [-0.2668,  0.1697,  0.0248, -0.4415],
        [-0.1928,  0.1195, -0.2840, -0.3927]])
第二个块中的权重: tensor([[[-0.4437, -0.3307, -0.0615, -0.2151]])
第二个块中的偏置: tensor([-0.2803])
```

5.2.2 参数初始化

内置初始化

首先调用内置的初始化器。下面的代码将所有权重参数初始化为标准差为 0.01 的高斯随机变量，且将偏置参数设置为 0。

```
def init_normal(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, mean=0, std=0.01)
        nn.init.zeros_(m.bias)
    net.apply(init_normal)
net[0].weight.data[0], net[0].bias.data[0]

(tensor([ 8.6460e-03,  2.7274e-03, -9.1820e-05,  4.2095e-03]), tensor(0.))
```


也可以定义 weight 为常量 1

```
def init_constant(m):
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight, 1)
        nn.init.zeros_(m.bias)
    net.apply(init_constant)
    net[0].weight.data[0], net[0].bias.data[0]

(tensor([1., 1., 1., 1.]), tensor(0.))
```

还可以对某些块应用不同的初始化方法。例如，使用 Xavier 初始化方法初始化第一个神经网络层，然后将第三个神经网络层初始化为常量值 42。

```
def init_xavier(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)
def init_42(m):
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight, 42)
net[0].apply(init_xavier)
net[2].apply(init_42)
print(net[0].weight.data)
print(net[2].weight.data)

tensor([[ 0.6197,  0.1737,  0.6839, -0.2637],
        [-0.6517,  0.5348,  0.3363, -0.6360],
        [-0.5237,  0.2189, -0.6857,  0.0790],
        [-0.2172,  0.1830, -0.0432, -0.3294],
        [-0.2473,  0.2120, -0.5750, -0.5354],
        [-0.0102,  0.1035, -0.2198,  0.4500],
        [-0.2974, -0.5293, -0.5277,  0.3527],
        [-0.3757, -0.0575, -0.6505, -0.3000]])
tensor([[42., 42., 42., 42., 42., 42., 42., 42.],
        [42., 42., 42., 42., 42., 42., 42., 42.]])
```

自定义初始化：

先定义 weight 为-10 到 10 的 uniform 均匀分布，然后通过一个乘法小技巧，让绝对值小于 5 的都为 0，这样就可以实现，-10 到-5 均匀分布的概率是 1/4，-5 到 5 为 0 的概率为 1/2，5 到 10 均匀分布的概率为 1/4

```
def my_init(m):
    if type(m) == nn.Linear:
        print("Init", *[(name, param.shape) for name, param in m.named_parameters()])
        nn.init.uniform_(m.weight, -10, 10)
        m.weight.data *= m.weight.data.abs() >= 0.5

    net.apply(my_init)
    net[0].weight[:2]

Init ('weight', torch.Size([8, 4])) ('bias', torch.Size([8]))
Init ('weight', torch.Size([2, 8])) ('bias', torch.Size([2]))

tensor([[ -5.9336,  3.1112, -0.5567, -9.5144],
        [ 6.8536, -1.7476, -7.3248,  7.5859]], grad_fn=<SliceBackward0>)
```

5.2.3 参数绑定

当需要在多个层间共享参数时，可以定义一个稠密层，然后使用它的参数来设置另一个层的参数。

参数绑定

```
shared = nn.Linear(8,8)
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(),
                    shared, nn.ReLU(),
                    shared, nn.ReLU(),
                    nn.Linear(8, 2), nn.ReLU())

net(X)
print(net[2].weight.data[0] == net[4].weight.data[0])
net[2].weight.data[0, 0] = 100
print(net[2].weight.data[0, 0] == net[4].weight.data[0, 0])

tensor([True, True, True, True, True, True, True, True])
tensor(True)
```

可以看到，在第三层和第 5 层的参数是一样的，且他们不是数值相等，而是同一个对象。

实验 3：自定义层，学习 5.4

5.4.1 不带参数的层

定义一个，功能为归一化的层，即每次用原本的数据减去一组数据的平均值

不带参数的层

```
class CenteredLayer(nn.Module):
    def __init__(self):
        super().__init__()
    def forward(self, Y):
        return Y - Y.mean()

layer = CenteredLayer()
layer(torch.FloatTensor([1,2,3,45,3]))

tensor([-9.8000, -8.8000, -7.8000, 34.2000, -7.8000])
```

可以将层作为组件合并到更复杂的模型中。

```
net = nn.Sequential(nn.Linear(6, 128), layer)
Y = net(torch.randn(3,6))
Y.sum()

tensor(2.6226e-06, grad_fn=<SumBackward0>)
```

可以看到，对于自定义的归一化层，对处理完成后的 y 求和，结果接近 0，代表定义的层是正确的。

5.4.2 带参数的层

如果需要实现自定义版本的全连接层，则该层需要两个参数，一个用于表示权重，另一个用于表示偏置项。在此实现中，可以使用修正线性单元作为激活函数。该层需要输入参数：in_units 和 units，分别表示输入数和输出数。

带参数的层

```
class MyLinear(nn.Module):
    def __init__(self, in_units, units):
        super().__init__()
        self.weight = torch.randn(in_units, units)
        self.bias = torch.randn(units)
    def forward(self, X):
        linear = torch.matmul(X, self.weight) + self.bias
        return F.relu(linear)
```

```
[8] ✓ 0.0s
```

```
> linear = MyLinear(5,3)
linear.weight
```

```
[9] ✓ 0.0s
```

```
'''
tensor([[ 0.1356,  1.5884, -1.3076],
        [ 0.3588, -0.0285, -0.7663],
        [-0.7685, -0.1405,  0.7614],
        [ 0.7127, -0.0929,  0.1339],
        [ 0.1547, -1.8781,  0.2164]])
```

可以使用自定义层直接执行前向传播计算，还可以使用自定义层构建模型，就像使用内置的全连接层一样使用自定义层。

```
linear(torch.randn(2,5))
```

```
✓ 0.0s
```

```
tensor([[2.1251, 1.4462, 0.0000],
        [1.8572, 2.9864, 0.0000]])
```

```
net = nn.Sequential(MyLinear(64, 8), MyLinear(8, 1))
net(torch.randn(3, 64))
```

```
✓ 0.0s
```

```
tensor([[7.3976],
        [5.4151],
        [6.8050]])
```

实验 4：读写文件，学习 5.5

5.5.1 加载和保存张量

对于单个张量，可以直接调用 `load` 和 `save` 函数分别读写它们。这两个函数都要求提供一个名称，`save` 要求将要保存的变量作为输入。

```
x = torch.arange(4)
torch.save(x, 'x_file')
```

```
[38] ✓ 0.0s
```

```
> x2 = torch.load('x_file')
```

```
[39] ✓ 0.0s
```

```
'''
tensor([0, 1, 2, 3])
```

可以看到在文件夹下生成了名为 `x_file` 的二进制文件
也可以以存储一个张量列表，然后把它们读回内存。

```

y = torch.zeros(4)
y
tensor([0., 0., 0., 0.])

torch.save([x, y], 'x_files')
x2, y2 = torch.load('x_files')
x2, y2
(tensor([0, 1, 2, 3]), tensor([0., 0., 0., 0.]))

```

可以写入或读取从字符串映射到张量的字典。相当于给每个张量一个名字映射，当要读取或写入模型中的所有权重时很方便。

```

mydict = {'x':x, 'y':y}
torch.save(mydict, 'mydict_file')
dict2 = torch.load('mydict_file')
dict2
{'x': tensor([0, 1, 2, 3]), 'y': tensor([0., 0., 0., 0.])}

```

5.5.2 加载和保存模型参数

深度学习框架提供了内置函数来保存和加载整个网络。但是只是保存模型的参数而不是保存整个模型。例如，如果有一个 3 层多层感知机，则需要单独指定架构。因为模型本身可以包含任意代码，所以模型本身难以序列化。因此，为了恢复模型，需要用代码生成架构，然后从磁盘加载参数。

加载和保存模型参数

```

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(8, 256)
        self.output = nn.Linear(256, 10)
    def forward(self, X):
        return self.output(F.relu(self.hidden(X)))

X = torch.randn(2, 8)
net = MLP()
Y = net(X)

torch.save(net.state_dict(), 'mlp_params')

clone = MLP()
clone.load_state_dict(torch.load('mlp_params'))
clone.eval()

MLP(
  (hidden): Linear(in_features=8, out_features=256, bias=True)
  (output): Linear(in_features=256, out_features=10, bias=True)
)

```

在文件夹下生成了 mlp_params 二进制文件，可以用 load_state_dict 加载到新

的对象中，且新的结果和原来的结果一样，X 是随机生成的，证明 clone 和 net 的参数是一样的，参数的加载是成功的

```
Y_clone = clone(X)
Y_clone == Y

tensor([[True, True, True, True, True, True, True, True, True, True],
        [True, True, True, True, True, True, True, True, True, True]])
```

实验 5：构建含有三个隐藏层的感知机网络对 Fashion-MNIST 的分类

构建感知机网络

通过以下设置构建：隐藏层神经元个数分别设为 256,128；

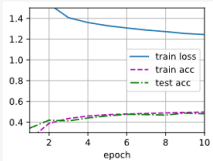
```
# net = FCClassification()
net = nn.Sequential(nn.Flatten(),nn.Linear(784, 256), nn.ReLU(), nn.Linear(256, 128), nn.ReLU(), nn.Linear(128, 10), nn.ReLU())
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
net.apply(init_weights)

Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): ReLU()
  (3): Linear(in_features=256, out_features=128, bias=True)
  (4): ReLU()
  (5): Linear(in_features=128, out_features=10, bias=True)
  (6): ReLU()
)
```

且采用交叉熵损失

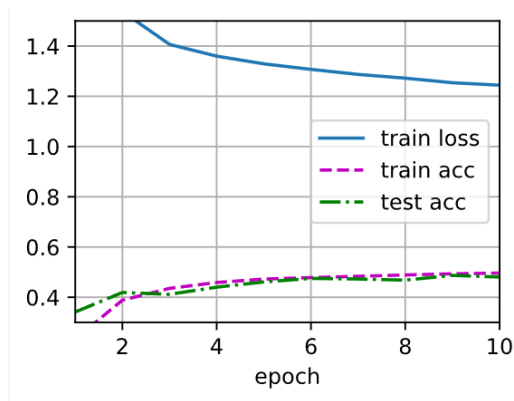
```
num_epochs = 10
time = d2l.Timer()
train_ch3(net, train_iter, test_iter, loss, num_epochs, trainer)
f'num_epochs = {num_epochs}, \n batch_size = {batch_size}, \n lr = {lr}, \n 用时: {time.stop():.2f} sec'

'num_epochs = 10, \n batch_size = 256, \n lr = 0.1, \n 用时: 70.67 sec'
```



画出实验结果

使用上一次实验课生成图像的代码，训练十轮后，可以得到结果，以下为训练损失、训练正确率、测试正确率随训练轮次 Epoch 的变化曲线

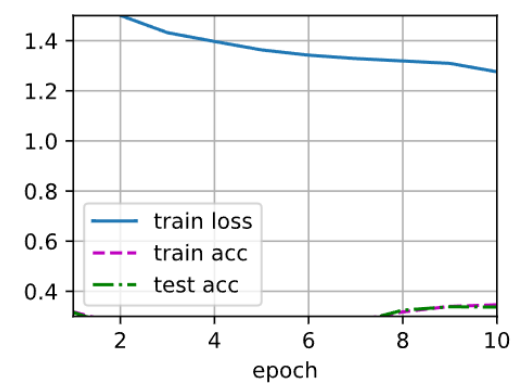


尝试不同的激活函数，观察和分析实验结果

尝试双曲正切激活函数：

```
# net = FNClassification()
net = nn.Sequential(nn.Flatten(),nn.Linear(784, 256), nn.Tanh(), nn.Linear(256, 128), nn.Tanh(), nn.Linear(128, 10), nn.Tanh())
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
net.apply(init_weights)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): Tanh()
  (3): Linear(in_features=256, out_features=128, bias=True)
  (4): Tanh()
  (5): Linear(in_features=128, out_features=10, bias=True)
  (6): Tanh()
)
```



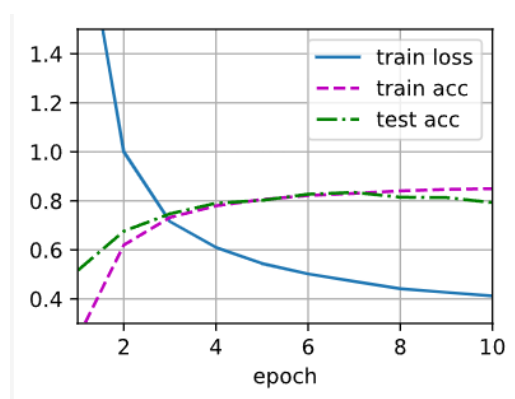
更换为双曲激活函数后，结果更差了。

更换成 LeakyReLU

```
# net = FNClassification()
net = nn.Sequential(nn.Flatten(),nn.Linear(784, 256), nn.LeakyReLU(), nn.Linear(256, 128), nn.LeakyReLU(), nn.Linear(128, 10), nn.LeakyReLU())
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
net.apply(init_weights)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): LeakyReLU(negative_slope=0.01)
  (3): Linear(in_features=256, out_features=128, bias=True)
  (4): LeakyReLU(negative_slope=0.01)
  (5): Linear(in_features=128, out_features=10, bias=True)
  (6): LeakyReLU(negative_slope=0.01)
)
```

结果为：效果非常理想，acc 高达 0.8 以上

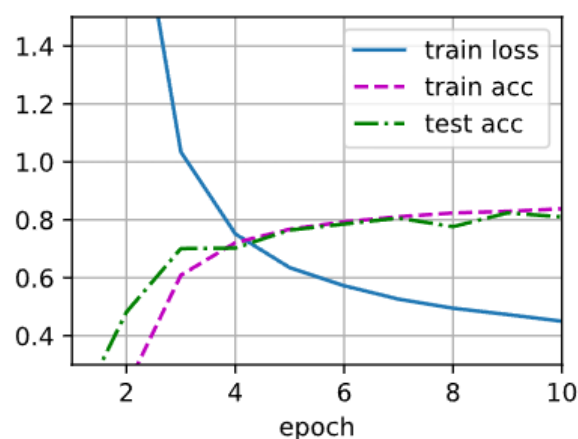


更换成 GELU（Gaussian Error Linear Unit）激活函数

```
# net = FNClassification()
net = nn.Sequential(nn.Flatten(),nn.Linear(784, 256), nn.GELU(), nn.Linear(256, 128), nn.GELU(), nn.Linear(128, 10), nn.GELU())
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
net.apply(init_weights)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): GELU(approximate='none')
  (3): Linear(in_features=256, out_features=128, bias=True)
  (4): GELU(approximate='none')
  (5): Linear(in_features=128, out_features=10, bias=True)
  (6): GELU(approximate='none')
)
```

结果为：分类效果非常理想，acc 高达 0.8 以上



在网络中加入 dropout

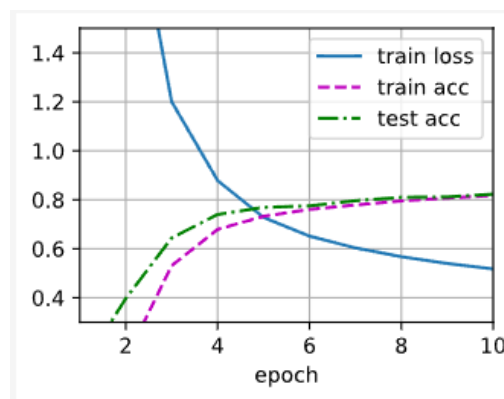
在神经网络中加入 Dropout 层是一种有效的正则化方法，可以防止模型过拟合。Dropout 的基本思想是在训练过程中随机丢弃（即置为零）一部分神经元的输出，从而增强模型的泛化能力。

添加概率为 0.5 的 Dropout

```
# net = FNClassification()
net = nn.Sequential(nn.Flatten(),nn.Linear(784, 256), nn.GELU(), nn.Dropout(0.5), nn.Linear(256, 128), nn.GELU(), nn.Dropout(0.5), nn.
Linear(128, 10), nn.GELU())
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
net.apply(init_weights)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): GELU(approximate='none')
  (3): Dropout(p=0.5, inplace=False)
  (4): Linear(in_features=256, out_features=128, bias=True)
  (5): GELU(approximate='none')
  (6): Dropout(p=0.5, inplace=False)
  (7): Linear(in_features=128, out_features=10, bias=True)
  (8): GELU(approximate='none')
)
```

结果为：和没有添加 Dropout 时相比，acc 的上升更加平稳，loss 变大

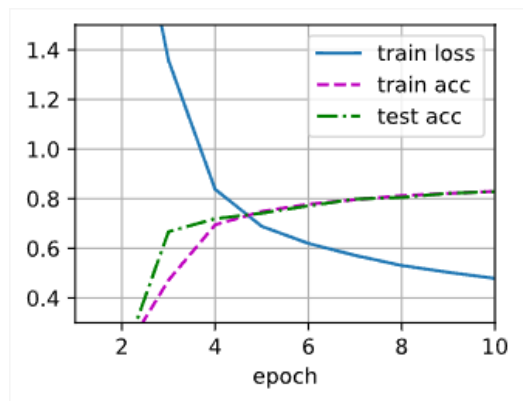


添加概率为 0.3 的 Dropout

```
# net = FNClassification()
net = nn.Sequential(nn.Flatten(),nn.Linear(784, 256), nn.GELU(), nn.Dropout(0.3), nn.Linear(256, 128), nn.GELU(), nn.Dropout(0.3), nn.
Linear(128, 10), nn.GELU())
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
net.apply(init_weights)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): GELU(approximate='none')
  (3): Dropout(p=0.3, inplace=False)
  (4): Linear(in_features=256, out_features=128, bias=True)
  (5): GELU(approximate='none')
  (6): Dropout(p=0.3, inplace=False)
  (7): Linear(in_features=128, out_features=10, bias=True)
  (8): GELU(approximate='none')
)
```

结果为：和之前的结果相比，第一轮的训练效果变差，cong 第二轮开始 acc 显著提高，loss 明显收敛，同样训练十轮后，acc 并没有提高。

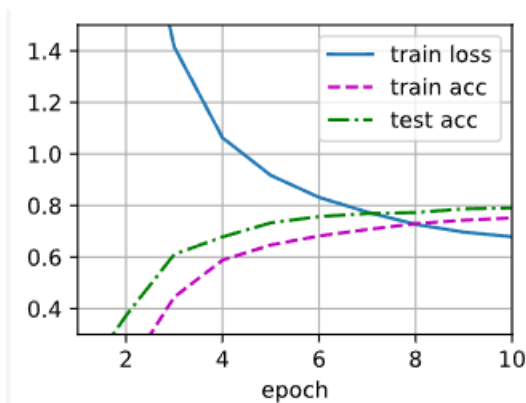


添加概率为 0.8 的 Dropout

```
# net = FNClassification()
net = nn.Sequential(nn.Flatten(),nn.Linear(784, 256), nn.GELU(), nn.Dropout(0.8), nn.Linear(256, 128), nn.GELU(), nn.Dropout(0.8), nn.
Linear(128, 10), nn.GELU())
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)
net.apply(init_weights)
```

```
Sequential(
  (0): Flatten(start_dim=1, end_dim=-1)
  (1): Linear(in_features=784, out_features=256, bias=True)
  (2): GELU(approximate='none')
  (3): Dropout(p=0.8, inplace=False)
  (4): Linear(in_features=256, out_features=128, bias=True)
  (5): GELU(approximate='none')
  (6): Dropout(p=0.8, inplace=False)
  (7): Linear(in_features=128, out_features=10, bias=True)
  (8): GELU(approximate='none')
)
```

结果为: 由于丢弃层设置的过大, 出现了训练 acc 和测试 acc 有显著差异的现象, 训练集和测试集的 acc 都有所降低, loss 增大, 结果变差。



[小结或讨论]

习题

习题 5.1.1

1. 如果将 MySequential 中存储块的方式更改为 Python 列表, 会出现什么样的问题?

可以通过以下代码把他改成普通列表，定义一个空的 bl 列表，每次把 linear 或者 relu 加进去，在前向传播函数中，再每个取出来，依次让执行到 X 上面，结果是正确的。

```
如果用普通的列表，而不用orderdict

class MySequential1(nn.Module):
    def __init__(self, *args):
        super().__init__()
        self.bl = []
        for idx, m in enumerate(args):
            self.bl.append(m)

    def forward(self, X):
        for block in self.bl:
            X = block(X)
        return X

net = MySequential1(nn.Linear(20, 256), nn.ReLU(), nn.Linear(256, 20))
net(X)

tensor([[ 0.3136, -0.2329, -0.0876, -0.0515, -0.0794,  0.2604,  0.1598, -0.0086,
          0.0131, -0.0353,  0.0669,  0.1358,  0.0756, -0.0200,  0.1264,  0.0946,
         -0.0689,  0.1392, -0.2528,  0.0427],
        [ 0.2238, -0.2446, -0.0876,  0.0013,  0.1505,  0.1832,  0.1944, -0.0491,
          0.1546, -0.1129,  0.1657,  0.2833,  0.1114, -0.0517,  0.1823, -0.0008,
         -0.1509,  0.2116, -0.3624,  0.0035],
        [ 0.3865, -0.2516, -0.0283, -0.0345, -0.0230,  0.1721,  0.1734, -0.0457,
          0.1138, -0.1436,  0.0076,  0.1489,  0.0376, -0.0341,  0.1207, -0.1327,
         -0.1234,  0.1920, -0.1734,  0.0058]], grad_fn=<AddmmBackward0>)
```

习题 5.1.2

2. 实现一个块，它以两个块为参数，例如 net1 和 net2，并返回前向传播中两个网络的串联输出。这也被称为平行块。

先定义两个子块，分别为 PML1，PML2，子块 1 中的操作为，20*256FC，ReLU，256*10FC，子块 2 中的操作为，10*256FC，ReLU，256*20FC，对于 3*10 的输入 X，依次经过两个网络后，输出应该为 3 * 20。实例化时，PML1，PML2 都作为平行 PML 的参数。

```

class PMPL1(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(20, 256)
        self.output = nn.Linear(256, 10)
    def forward(self, X):
        return self.output(F.relu(self.hidden(X))) # 先hidden, 再relu, 再out

class PMPL2(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(10, 256)
        self.output = nn.Linear(256, 20)
    def forward(self, X):
        return self.output(F.relu(self.hidden(X))) # 先hidden, 再relu, 再out

class ParallerMLP(nn.Module):
    def __init__(self, net1, net2):
        super().__init__()
        self.net1 = net1
        self.net2 = net2
    def forward(self, X):
        pmp1 = self.net1()
        pmp2 = self.net2()
        X = pmp1(X)
        X = pmp2(X)
        return X

```

调用，输出是正确的。

```

parnet = ParallerMLP(PMPL1,PMPL2)
parnet(X)

```

✓ 0.0s

```

tensor([[ -7.1413e-02, -3.4688e-02,  1.1177e-01,  1.1232e-01, -3.7961e-02,
          -1.4884e-01,  3.0932e-02, -1.0127e-01,  7.1531e-02,  1.3204e-02,
           5.1400e-02,  1.7620e-02, -1.9414e-02, -2.4954e-02,  4.8306e-02,
           2.9852e-02, -7.1433e-02,  3.0474e-02, -7.3710e-02,  3.4978e-02],
        [-3.6647e-02, -3.1103e-02,  1.1975e-01,  9.4366e-02, -3.8415e-02,
          -1.5131e-01,  2.5494e-02, -7.3312e-02,  8.5471e-02,  3.4098e-03,
           4.4314e-02,  3.8692e-03,  1.2084e-02, -1.5273e-02,  3.5809e-02,
           1.7297e-05, -7.4243e-02,  3.8184e-02, -9.5779e-02,  3.0458e-02],
        [-5.8447e-02, -3.0578e-02,  1.0993e-01,  1.4666e-01, -3.5515e-02,
          -1.5797e-01,  2.7435e-02, -7.6945e-02,  8.6590e-02,  4.7143e-02,
           1.4770e-02,  3.2651e-02,  5.7115e-03, -5.5368e-02,  4.5711e-02,
           1.0798e-02, -3.5756e-02,  3.9832e-02, -6.5031e-02,  1.0843e-02]],
        grad_fn=<AddmmBackward0>)

```

习题 5.2.1

1. 使用 5.1 节中定义的 MLP 模型，访问各个层的参数。

MLP 中没有定义 net 的层数，但是可以直接通过自定义的属性 hidden 和 output 来查看每个的参数。

```

class MLP(nn.Module):
    # 模型参数声明层，这里我们声明两个全连接层
    def __init__(self):
        # 调用父类Module的初始化函数来必要的初始化，这样在类实例化时也可以指定别的参数
        super().__init__()
        self.hidden = nn.Linear(4, 256)
        self.output = nn.Linear(256, 4)
    def forward(self, X):
        return self.output(F.relu(self.hidden(X))) # 先hidden, 再relu, 再out
net = MLP()
net(X)

tensor([[ 0.1313, -0.1307, -0.0266, -0.2639],
        [ 0.0358, -0.0358,  0.1625, -0.2142]], grad_fn=<AddmmBackward0>)

print(f'隐藏层的形状: {net.hidden.weight.data.shape}')
print(f'隐藏层第一行: {net.hidden.weight.data[0]}')
print(f'输出的形状: {net.output.weight.data.shape}')
print(f'输出层: {net.output.weight.data}')

隐藏层的形状: torch.Size([256, 4])
隐藏层第一行: tensor([ 0.0657,  0.2765, -0.2024,  0.4044])
输出的形状: torch.Size([4, 256])
输出层: tensor([[ 0.0275, -0.0529, -0.0286, ...,  0.0617, -0.0605,  0.0570],
                 [-0.0518,  0.0308,  0.0077, ..., -0.0544, -0.0578, -0.0237],
                 [ 0.0553,  0.0347, -0.0234, ..., -0.0130, -0.0018,  0.0085],
                 [-0.0220,  0.0295, -0.0565, ...,  0.0095, -0.0339, -0.0423]])

```

习题 5.4.1

1.设计一个接受输入并计算张量降维的层，它返回 $y_k = \sum_{i,j} W_{ijk} x_i x_j$

具体降维的方法为，先 X 乘以其转置，再乘以 W 对应的维度的二维向量，再求和，对应的得到的 y 为一维向量

```

class Squee(nn.Module):
    def __init__(self, W):
        super().__init__()
        self.wi = W.shape[0]
        self.wj = W.shape[1]
        self.wk = W.shape[2]
        self.W = W
    def forward(self, X):
        y = torch.randn(self.wk,1)
        for i in range(self.wk):
            y[i] = torch.sum(torch.mul(torch.matmul(X, X.T), self.W[:, :, i]))
        return y

```

结果为:

```

W = torch.randn(size=(3,3,3))
sq = Squee(W)
X = torch.randn(3,1)
sq(X)

tensor([[ -0.8416],
        [ -0.2401],
        [  0.5914]])

```

习题 5.5.2

2. 假设我们只想复用网络的一部分，以将其合并到不同的网络架构中。比如想在一个新的网络中使用之前网络的前两层，该怎么做？

设计一个，第一个网络中有三层，分别为：2 * 10， 10 * 20， 20 * 4

第二个网络使用第一个网络的前两层，第三层为：20 * 5

当输入 X 为 10 * 2 时，如果复用成功，则会输出 10 * 5 的 tensor

先定义第一个 Net

```
class FirstNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(2, 10)
        self.fc2 = nn.Linear(10, 20)
        self.fc3 = nn.Linear(20, 4)

    def forward(self, x):
        x = self.fc1(x)
        x = self.fc2(x)
        x = self.fc3(x)
        return x
```

实例化之后保存和加载参数

```
model = FirstNet()
X = torch.randn(10, 2)
model(X)
first_two_layers = nn.Sequential(
    model.fc1,
    model.fc2
)

torch.save(model.state_dict(), 'model_params')

model.load_state_dict(torch.load('model_params'))

<All keys matched successfully>
```

定义第二个 Net

```
class SecondNet(nn.Module):
    def __init__(self, first_two_layers):
        super().__init__()
        self.first_two_layers = first_two_layers
        self.fc2 = nn.Linear(20, 5)

    def forward(self, x):
        x = self.first_two_layers(x) # 使用第一个模型的前两层
        x = self.fc2(x)
        return x
```

实例化

```
new_model = SecondNet(first_two_layers)
print(new_model)

SecondNet(
  (first_two_layers): Sequential(
    (0): Linear(in_features=2, out_features=10, bias=True)
    (1): Linear(in_features=10, out_features=20, bias=True)
  )
  (fc2): Linear(in_features=20, out_features=5, bias=True)
)
```

输出为 $10 * 5$

```
new_model(X)

tensor([[ 0.0179, -0.0409,  0.0351,  0.1176, -0.4664],
        [ 0.1088,  0.0209,  0.1162,  0.2193, -0.6036],
        [-0.1191, -0.1036, -0.1101, -0.0401, -0.2328],
        [ 0.1345,  0.0188,  0.1539,  0.2509, -0.6597],
        [-0.0524, -0.0045, -0.0911,  0.0264, -0.2860],
        [-0.0795, -0.1369, -0.0294,  0.0132, -0.3457],
        [-0.0405,  0.0097, -0.0852,  0.0387, -0.2985],
        [-0.0867, -0.1500, -0.0296,  0.0064, -0.3421],
        [ 0.0075,  0.0779, -0.0691,  0.0870, -0.3395],
        [ 0.0682, -0.0076,  0.0807,  0.1741, -0.5432]],
        grad_fn=<AddmmBackward0>)
```

为了进一步验证参数的共享，可以运行以下代码，观察到，第一层的 $2 * 10$ 参数时相等的

```
model.fc1.weight.data == new_model.first_two_layers[0].weight.data

tensor([[True, True],
        [True, True],
        [True, True],
        [True, True],
        [True, True],
        [True, True],
        [True, True],
        [True, True],
        [True, True],
        [True, True]])
```

第二层的 $10 * 20$ 也是相等的

在实验 5 中，通过尝试不同的激活函数和加入 Dropout 层，观察到了对模型性能的影响。例如，使用 LeakyReLU 和 GELU 激活函数时，模型的准确率显著提高，这表明合适的激活函数可以提高模型的非线性拟合能力。加入 Dropout 层可以有效防止模型过拟合，但过高的丢弃概率会导致模型性能下降。

4. 参数共享与网络复用

在做题时尝试了将一个网络的部分层复用到另一个网络中。虽然在同一个程序运行中，两个网络的前两层参数值相等，但它们并不是同一个对象。由于形状不一致，目前没有找到很好的方法让两个网络公用一个对象。